



Data Structures and Algorithms -3 (25CS2103E)

IntelliSpell: Intelligent Spell Checker using Levenshtein Distance

Team Members

S. Manvi Vaishnavi: 2520030326

Rohini Tanmayi D: 2520030349

Pramod Krishna M: 2520030111

Under the Esteemed Guidance of

DR. CH Anuradha

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

IntelliSpell is a browser-based intelligent spell checker built on the Levenshtein edit distance algorithm. It splits text into words, checks each one against a dictionary of 9,844 words, and for every unknown word ranks up to three suggestions within an edit distance of two.

Users can type or paste text, or open a .txt, .pdf or .docx file. Misspelled words are underlined; clicking one opens its suggestions and the chosen word replaces the error right there in the text. The corrected text can be copied or downloaded.

All processing happens locally in the browser, so no document leaves the device. An interactive matrix view shows how each distance is calculated, making the algorithm transparent as well as fast.

Introduction

- **Spelling errors** such as recieve, seperate and definately are easy to make and easy to miss.
- **Spell checking** means finding a word that is not valid and suggesting the intended one.
- **Edit distance** measures how different two words are by counting single-letter insertions, deletions and substitutions.
- **IntelliSpell** applies this idea in the browser: type text or open a file, click a red word and fix it instantly.

Levenshtein distance

Fewest single-letter edits to turn one word into another. Every edit costs 1.

Insert = 1 Delete = 1 Substitute = 1

Literature Survey

Work	Contribution	Use in IntelliSpell
Damerau (1964)	Studied common spelling mistakes such as adding, removing, changing, or swapping letters.	Helps us handle common typing mistakes, especially swapped letters like “teh” → “the”.
Levenshtein (1965/66)	Introduced the idea of measuring how many insertions, deletions, and substitutions are needed to change one word into another.	We use Levenshtein distance to find and rank similar words.
Burkhard & Keller (1973)	Proposed the BK-tree , which helps search for words that are close to a given word using a distance measure.	Can be used later to make IntelliSpell faster when the dictionary is very large.
Wagner & Fischer (1974)	Developed a dynamic-programming method for finding the edit distance between two strings.	Our project uses the matrix/dynamic-programming approach to calculate word similarity.
Kukich (1992)	Reviewed different methods used for automatically detecting and correcting spelling errors.	Helps us understand how dictionary-based and edit-distance methods are used in spell checkers.
Norvig (2007)	Demonstrated a simple spelling corrector that generates possible words within a small edit distance.	Our project follows a similar basic idea: dictionary + small edit distance → possible corrections .

Problem Statement and Objectives

Problem Statement

Typing mistakes are easy to make and easy to miss. A useful checker must spot the wrong word, measure how far it is from real words, and suggest the right one, quickly and without sending private text to a server.

Accuracy

Find the intended word, not just any word

Privacy

Documents never leave the device.

Speed

Feel instant, even on long text.

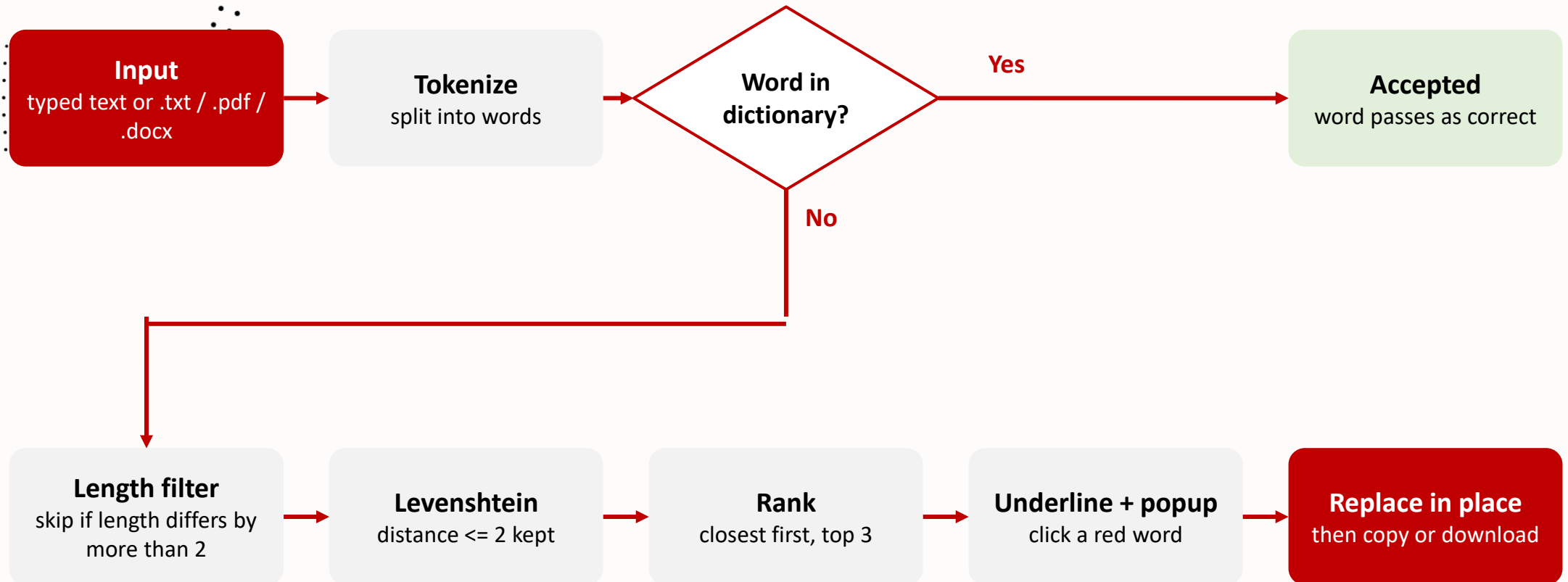
Flexibility

Typed text and common file formats.

Objectives

- 1 Detect misspelled words by checking every word against a dictionary.
- 2 Measure similarity between words using Levenshtein distance.
- 3 Rank the closest suggestions: up to three, within distance 2.
- 4 Let the user correct a word in place with one click.
- 5 Support typed text and .txt, .pdf and .docx files, and export the result.
- 6 Keep all processing in the browser to protect privacy.
- 7 Make the algorithm visible with an interactive matrix.

Flow Chart



RESULTS

Typed	Top suggestions	Expected	Match
recieve	receive, believe, recipe	receive	Yes – 1st
seperate	separate, operate, generate	separate	Yes – 1st
teh	the, tech, tel	the	Yes – 1st
wrod	word, wood, rod	word	Yes – 1st
freind	friend, find, trend	friend	Yes – 1st
wierd	weird, wired, were	weird	Yes – 1st
neccessary	necessary	necessary	Yes – 1st
enviroment	environment, environments	environment	Yes – 1st

20 / 20
Correct word came first

20 / 20
Correct word was in the top

~7 ms
Average time for one misspelled word

Hardware and Software requirements

Hardware

- Any laptop, desktop or phone
- Minimum 4 GB RAM, dual-core processor
- No GPU needed
- No server or database needed
- Internet only for first PDF/DOCX use (reader libraries load from a CDN)

Software

- HTML5, CSS3 and JavaScript (no framework, no backend)
- pdf.js 3.11.174 for .pdf files, mammoth 1.6.0 for .docx files
- IBM Plex Serif, Sans and Mono fonts
- Browser: Chrome, Edge or Firefox
- VS Code with Live Server, or Python http.server, to run it locally

Conclusion

Accurate

Edit distance finds the closest real words; 20 of 20 test typos ranked correctly first.

Private

Text and files are processed in the browser and never uploaded.

Fast

A hash-set lookup and a length filter keep checks near-instant.

Transparent

The matrix view shows exactly how each distance is worked out.

A classic algorithm, applied well, gives precise, private and explainable spell checking with no server required

Future Scope

Full Damerau-Levenshtein

count swaps as one edit everywhere, not only in ranking

Bigger, multilingual dictionaries

plus a saved personal dictionary

Context-aware checking

catch real-word errors such as there vs their

Faster search

BK-tree or trie instead of scanning by length

OCR for scanned PDFs

read image-only documents

Keep original format

export corrected .docx or .pdf, not only .txt

Proposed enhancements, not features of the current build.

Thank you